بنية if والاختيار الشرطي

الدرس 8 — 1IOT، SST-ICT-01 — برامج تقرر ماذا تفعل: if، if-else، التداخل

29/09/2026

1 مراجعة: printf و scanf

ملاحظة

مراجعة سريعة للدرس السابق. **printf** ينتج ما يسمى **الإخراج**: وفي حالتنا يذهب الإخراج إلى الشاشة. و**scanf** يسمح بالحصول على **إدخال**: وجهاز **الإدخال** لدينا هو لوحة المفاتيح. إذا تخيلنا البرنامج **صندوقًا**، فإننا نستطيع أن نجعل الصندوق ينتج شيئًا يخرج ويُعرض على المستخدم، وأن ندخل معلومات إلى الصندوق.

تنبيه

هذا المفهوم (صندوق يستقبل **إدخالًا** ويعطي إخراجًا) مهم جدًا لأنه سيعود باستمرار عند استخدام **Arduino**: فـArduino لوحة يمكننا أن نرسل إليها معلومات وأن نحصل منها على معلومات، بحسب ما نفعله بها.

تعريف

ملخص الصيغة:

```
// يطبع؛ %d للأعداد الصحيحة، %f لـfloat
printf("testo %d", variabile);
// يقرأ من لوحة المفاتيح؛ لا تنسَ & أبدًا
scanf("%d", &variabile);
```

printf بعد انتهاء الطباعة لا ينتقل **أبدًا** إلى سطر جديد من تلقاء نفسه: نحتاج إلى \n إذا أردنا ذلك صراحة. وscanf يقبل الأعداد فقط (**أبدًا** نصًا حرًا، فهو ليس ذكاءً اصطناعيًا للمحادثة): %d لعدد **صحيح**، و%f لعدد **بفاصلة عشرية**.

مثال

تمرين **مساحة** المربع، الذي حُل سابقًا ونأخذه نقطة انطلاق لهذا الدرس:

```
int L;
printf("Inserisci il lato del quadrato: ");
scanf("%d", &L);

int area;
area = L * L;

printf("L'area del quadrato vale %d", area);
```

2 كيف تنفذ الآلة: سلسلة من العمليات

تعريف

تنفذ الآلة كود المثال السابق **كسلسلة مرتبة من العمليات**، دائمًا بالترتيب نفسه:

1. تنشئ **المتغير** `L` (`int L;`)
2. تطبع الرسالة (**`printf`**)
3. تحصل على المعلومة من **لوحة المفاتيح** (`scanf`)
4. تنشئ **المتغير** `area` (`int area;`)
5. تحسب `L * L`
6. تطبع النتيجة (**`printf`**)

فالبرنامج يُعرَّف بأنه **قائمة مرتبة من العمليات**: إذا نُفذ 42 مرة فسينفذ دائمًا هذه السلسلة نفسها فقط، و**أبدًا** شيئًا مختلفًا.

تنبيه

المشكلة: البرنامج الذي يفعل دائمًا الشيء نفسه ليس مفيدًا. تشبيه استُخدم في القسم: لعبة فيديو فعلها الوحيد الممكن هو نفسه دائمًا ("اضغط هنا للبدء"، نهاية اللعبة) لن تكون ممتعة ولا مفيدة. نحتاج إلى برنامج قادر على **اختيار ما يفعله** بحسب الحالة.

3 المخطط الانسيابي: التسلسل مقابل الاختيار

ملاحظة

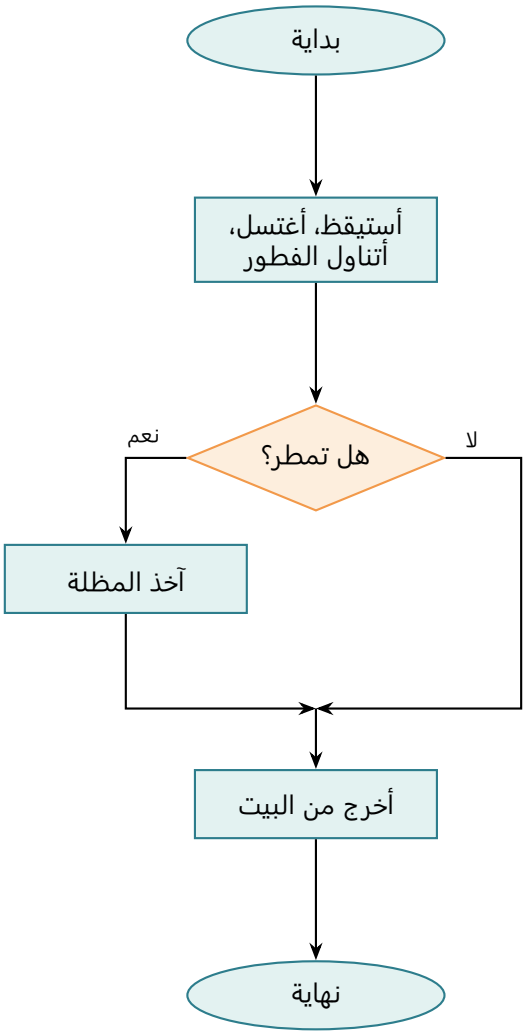
التمثيل البياني المستخدم في القسم: **المستطيل** يمثل فعلًا يجب القيام به (مثل إنشاء **متغير** أو طباعة رسالة). و**سلسلة** من المستطيلات موصولة بأسهم تمثل التدفق الخطي للعمليات، مثل مراحل لعبة فيديو: الأولى، الثانية، الثالثة... دائمًا بالترتيب نفسه.

تعريف

لتمثيل **اختيار** في التدفق نحتاج إلى رمز جديد، هو **المعيَّن** (**rombo**): نكتب داخله سؤالًا جوابه **صحيح**/خطأ (نعم/لا). ومن **المعيَّن** يخرج سهمان، واحد لكل جواب ممكن، يؤديان إلى مسارين مختلفين في البرنامج.

مثال

تشبيه من الحياة اليومية استُخدم لتقديم المفهوم: حمل المظلة دائمًا، مطرًا أو شمسًا، ليس خطأ لكنه مزعج. الأفضل: "أستيقظ، أغتسل، أرتدي ملابسي، أتناول الفطور (دائمًا على أي حال). أنظر إلى الخارج: إذا كانت تمطر آخذ المظلة؛ وإذا لم تكن تمطر أتركها في البيت." يسمى هذا **بنية القرار** (**costrutto decisionale**): تسمح بإجراء **اختيارات**، أي جعل التدفق يتبع مسارًا أو آخر بحسب **شرط** (**condizione**).



شكل 1: مخطط انسيابي مرسوم في القسم: المعيَّن "هل تمطر؟" يوزع التدفق على مسارين مختلفين، يلتقيان قبل المتابعة.

4 بنية if

تعريف

`if` (بالإنجليزية "إذا") هي **البنية** (**costrutto**) التي تترجم معيَّن **المخطط الانسيابي** إلى كود:

```
if (condizione)
{
    // كود يُنفذ فقط إذا كان الشرط صحيحًا
}
```

القوسان المعقوفان إلزاميان: ينشئان حاوية تحدد بالضبط أي التعليمات تنتمي إلى **الكتلة** "تُنفذ إذا كان **صحيحًا**" وأيها الاستمرار العادي للكود. بدون **الأقواس** لا تستطيع الآلة التمييز بينهما.

ملاحظة

`if` ليست مجرد **دالة** من مكتبة C: إنها **بنية** **من** **بنى** **اللغة**، أي كلمة مفتاحية موجودة في كل لغات البرمجة النصية تقريبًا (Python وC# وDelphi/Object Pascal...): فالاختيارات الشرطية حاجة أساسية إلى درجة أن غيابها استثناء نادر جدًا. أما عن PLC، فقد بيّن الأستاذ مسبقًا أننا عند دراستها سنجد طريقة للتعبير عن **شروط** مشابهة: في لغة الملامسات (**ladder logic**)، وهي الأكثر تقليدية في PLC، لا توجد كلمة `if` حقيقية – بل يُحصل على المنطق الشرطي نفسه بتركيب الملامسات والملفات على المخطط السلمي؛ أما لغات PLC الأحدث مثل **Structured Text** ففيها `if` نصية تشبه تمامًا ما في C.

تعريف

يُبنى **الشرط** بين القوسين باستخدام **العوامل** **الشرطية** التي رأيناها في الدروس السابقة (مثل <، >، =>، `==`): وهي تعبيرات قيمتها دائمًا صحيح أو خطأ، و**أبدًا** شيء بينهما.

5 مثال موجَّه: درجة طالب

مثال

مثال طُور مباشرة في القسم، مع تنفيذ حقيقي على مترجم عبر الإنترنت: برنامج يطلب درجة (**voto**، من عشرة، قيمة **صحيحة**) طالب. النسخة الأولى المكتوبة مباشرة، تُطبع فيها الرسالة الإيجابية دائمًا، وعند الدرجة المنخفضة فقط تضاف رسالة أخرى بـ`if`:

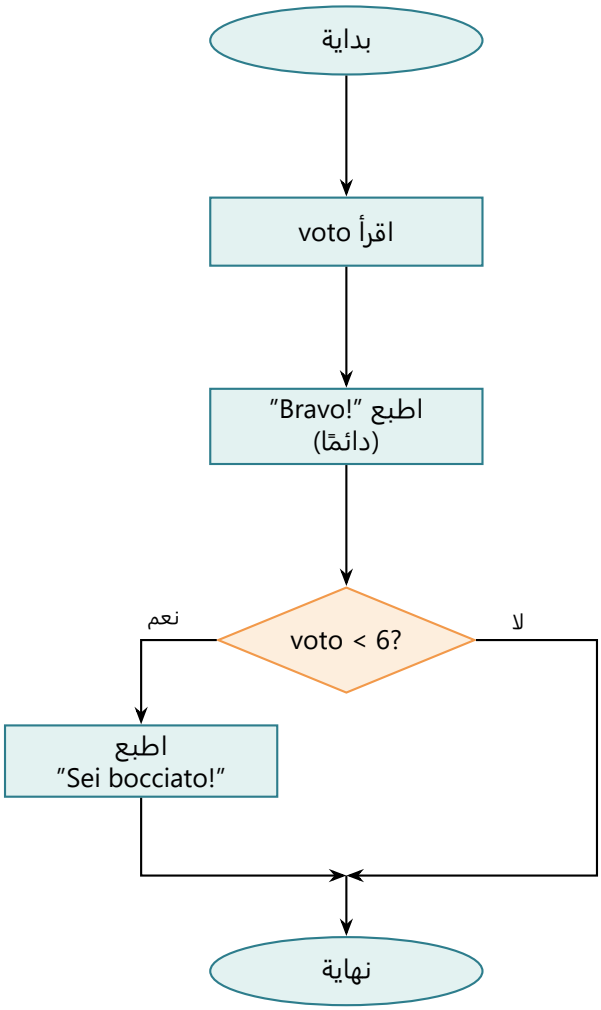
```c
int voto;
printf("Inserisci il voto dell'allievo: ");
scanf("%d", &voto);

printf("Bravo!");

if (voto < 6)
{
    printf("Sei bocciato!");
}
```

تنبيه

الخطأ الذي اكتشفناه معًا في القسم (**Bravo** = أحسنت، **Sei bocciato** = أنت راسب): مع الدرجة 8 يطبع البرنامج بشكل صحيح "Bravo!" فقط. لكن مع الدرجة 3 يطبع `"Bravo!Sei bocciato!"`، أي الرسالتين **كلتيهما**: فأول `printf("Bravo!")` يُنفذ على أي حال، لأنه خارج `if`، فهو تعليمة عادية في **التسلسل**، غير مرتبطة بأي **شرط**. `if` وحدها يمكنها فقط **إضافة** تعليمة عندما يكون **الشرط** صحيحًا: ولا يمكنها **استبدال** أو **منع** تعليمة مكتوبة خارجها. تنقصنا طريقة لنقول صراحة ماذا نفعل **وإلا** (أي بدلًا من طباعة "Bravo!").



شكل 2: مخطط انسيابي للنسخة الأولى (مع الخطأ): طباعة "Bravo!" تُنفذ دائمًا، أيًا كان فرع if، لأنها خارج الكتلة الشرطية.

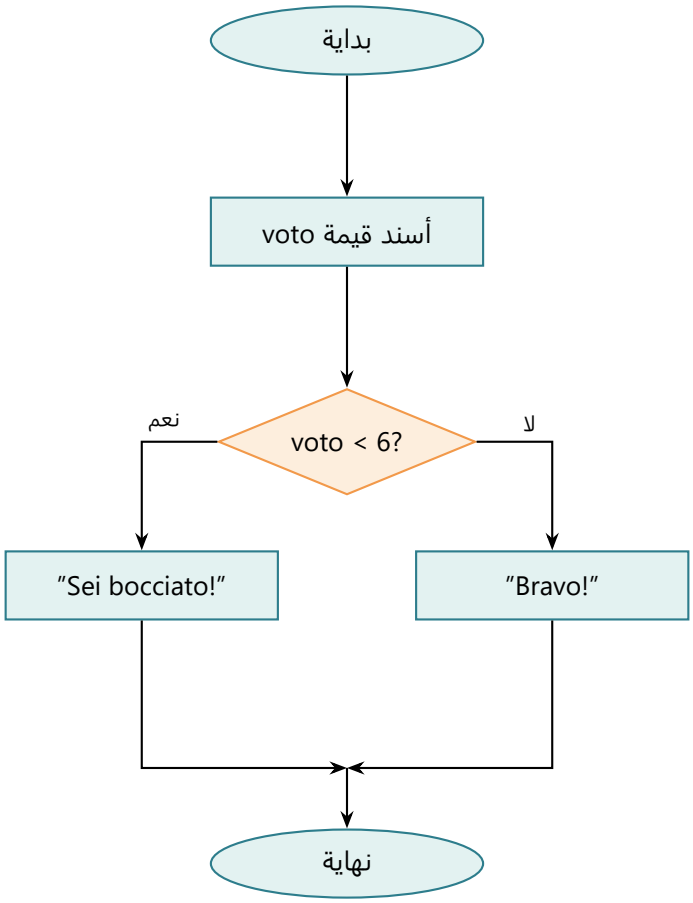
6 الشكل if-else

تعريف

`if…else` تضيف **الفرع** (**ramo**) "وإلا": إذا كان الشرط **صحيحًا** تُنفذ الكتلة الأولى، وإذا كان **خاطئًا** تُنفذ **كتلة** `else`. الكتلتان تستبعد إحداهما الأخرى دائمًا: تُنفذ واحدة فقط منهما.

```
int voto;
printf("Inserisci il voto dell'allievo: ");
scanf("%d", &voto);

if (voto < 6)
{
    printf("Sei bocciato!");
}
else
{
    printf("Bravo!");
}
```



شكل 3: مخطط انسيابي لـ`if-else`: الفرعان يستبعد أحدهما الآخر (تُنفذ دائمًا كتلة واحدة فقط)، ثم يلتقي التدفق قبل المتابعة.

مثال

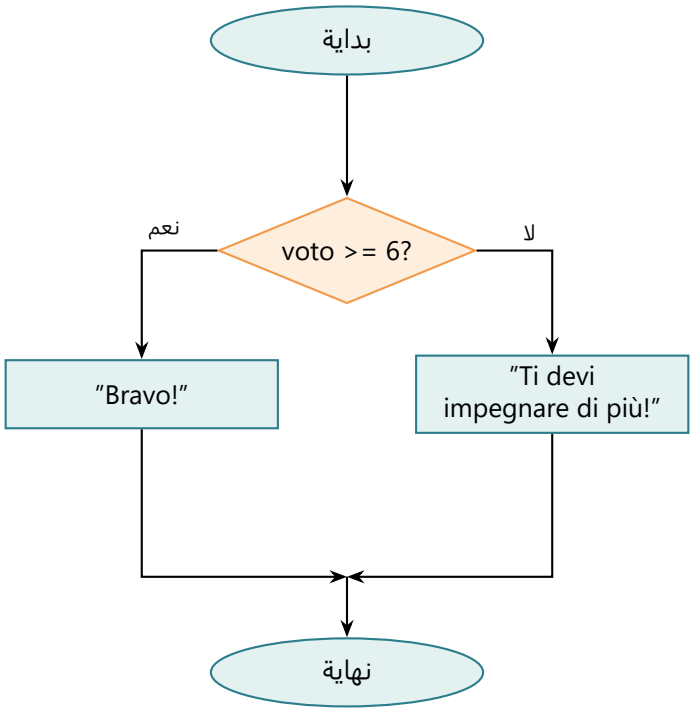
اختبارات أُجريت مباشرة في القسم على البرنامج أعلاه: مع الدرجة 8 يطبع البرنامج "Bravo!"؛ ومع الدرجة 3 يطبع "Sei bocciato!". يتغير سلوك البرنامج فعلًا بحسب ما يدخله المستخدم: لم يعد **تسلسلًا** ثابتًا.

ملاحظة

يمكن كتابة المسألة نفسها "بالعكس"، **بعكس** الشرط والرسائل، والنتيجة مطابقة:

```c
if (voto >= 6)
{
    printf("Bravo!");
}
else
{
    // راسب
    printf("Ti devi impegnare di piu'!");
}
```

للحصول على درجة **بفاصلة عشرية** يكفي تغيير **محدد التنسيق** إلى `scanf("%f", &voto);`، مع تعريف `voto` على أنه `float`: أما باقي منطق **if-else** فلا يتغير.



شكل 4: مخطط انسيابي للنسخة "المعكوسة": منطق if-else السابق نفسه، مع عكس الشرط والرسائل، والنتيجة مطابقة (**Ti devi impegnare di più** = عليك أن تجتهد أكثر).

7 انتبه إلى الحدود: أكبر مقابل أكبر أو يساوي

تنبيه

خطأ شائع جدًا: كتابة `if (voto > 6)` عندما نريد في الواقع اعتبار 6 درجة نجاح أيضًا. الآلة **صارمة**، ولا تفسر النيات: إذا كانت قيمة voto هي 6 بالضبط، فإن **الشرط** `voto > 6` **خاطئ** (6 ليس أكبر من 6)، وبـ`if` مكتوبة هكذا يصبح الطالب الحاصل على 6 "راسبًا". ولتضمين القيمة الحدية نحتاج إلى العامل **أكبر أو يساوي**: `voto >= 6`. **الخطأ** في هذه الحالات دائمًا خطأ المبرمج، لا الآلة: يجب كتابة **الشرط** بدقة.

8 التداخل: أكثر من احتمالين

تعريف

عندما تكون الاحتمالات المطلوب تمييزها أكثر من اثنين (ليس فقط "راسب/ناجح" بل مثلًا أربع فئات تقييم)، نستخدم **التداخل** (**annidamento**): `if` داخل `if` أخرى، بقدر ما نحتاج من المستويات. وكل `if` **متداخلة** تحتفظ بقوسيها الإلزاميين.

مثال

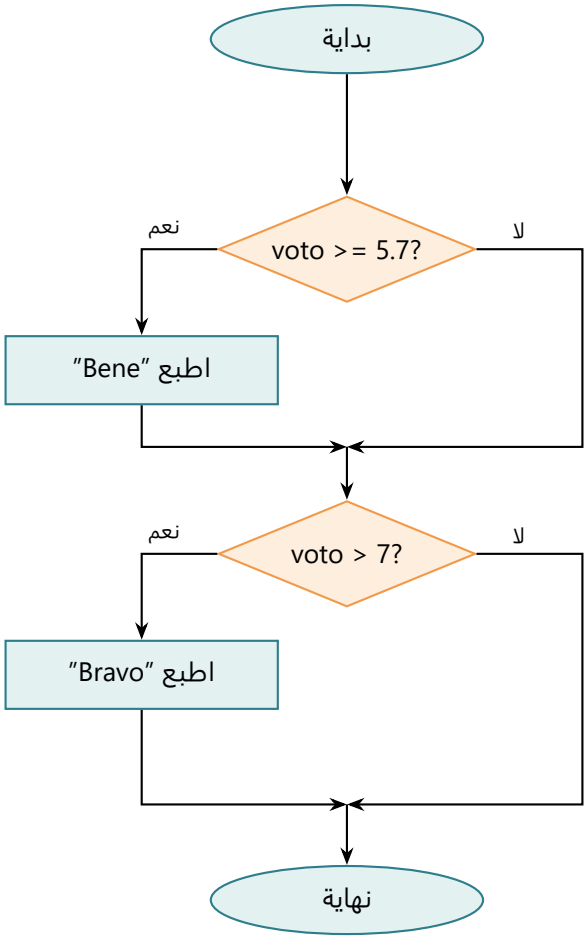
هدف نوقش في القسم: بناء طبقات من التقييم للدرجة — تحت 5 "راسب"، بين 5 و6 "عليك أن تجتهد أكثر"، بين 6 و7 "أحسنت"، فوق 7 "عبقري". تحتاج كل فئة إلى **شرط** خاص، يُبنى بانتباه إلى كيفية تحديد الفئات بعضها لبعض.

تنبيه

خطأ كلاسيكي يجب تجنبه: كتابة `if` **متتاليتين** بدلًا من **متداخلتين**، عندما لا تستبعد **الشروط** بعضها بعضًا. مثال نوقش في القسم:

```
// غير متداخل: if منفصلة
if (voto >= 5.7)
{
    printf("Bene");
}
// غير متداخل: if منفصلة
if (voto > 7)
{
    printf("Bravo");
}
```

مع الدرجة 8 يطبع هذا الكود "Bene" و"Bravo" **كلتيهما**، لأن 8 يحقق الشرطين المستقلين: وهذا ليس السلوك المطلوب إذا أردنا رسالة واحدة لكل درجة.



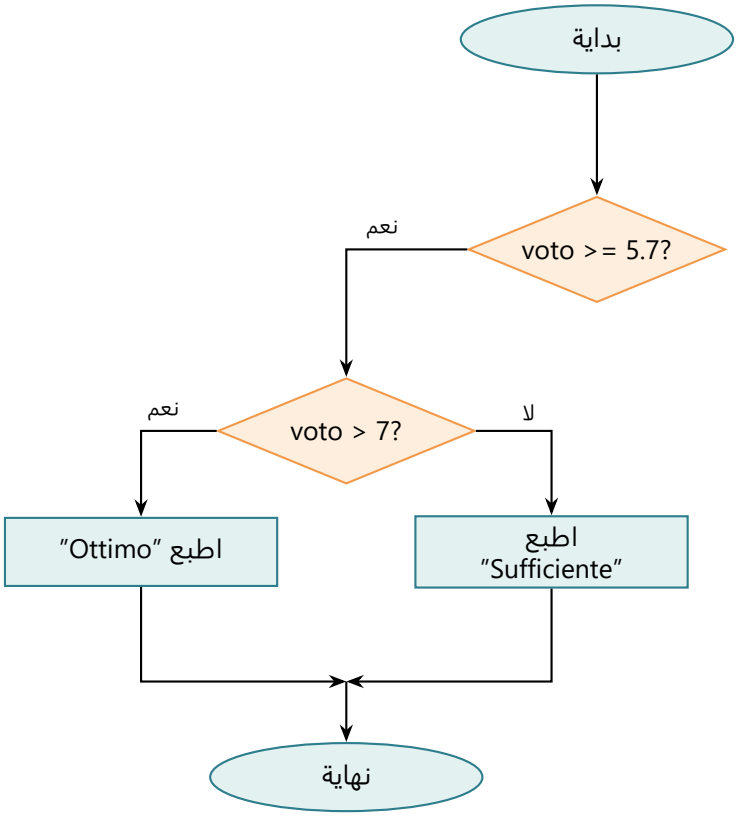
شكل 5: مخطط انسيابي للخطأ: `if` المتتاليتان (غير المتداخلتين) تُفحصان دائمًا، أيًا كانت نتيجة الأولى – مع الدرجة 8 يُطبع "Bene" و"Bravo" معًا.

تعريف

الحل الصحيح بالتداخل: تُكتب `if` الثانية داخل كتلة الأولى، كي تُفحص **فقط** **إذا** تأكدنا أن الشرط الأول **صحيح**:

```
if (voto >= 5.7)
{
    if (voto > 7)
    {
        printf("Ottimo");
    }
    else
    {
        printf("Sufficiente");
    }
}
```

بهذه البنية: إذا كانت قيمة voto هي 5.7 أو 6 أو 6.9 يطبع البرنامج "Sufficiente" (دخل **if** الأولى لا الثانية)؛ وإذا كانت 7.1 يطبع "Ottimo" (يحقق **الشرطين** معًا)؛ وإذا كانت تحت 5.7 فلا يدخل **الكتلة** أصلًا (في هذا المثال المبسط لا تُطبع أي رسالة لتلك الحالة، ويجب معالجتها بـ**else** خارجية).



شكل 6: مخطط انسيابي للحل الصحيح بالتداخل: `if` الثانية (voto > 7?) تُفحص فقط إذا كانت الأولى صحيحة – وهكذا لا تُطبع رسالتان معًا أبدًا. فرع "لا" من if الأولى (الدرجة تحت 5.7) غير ممثل: في هذا المثال المبسط لا تُطبع أي رسالة لتلك الحالة.

9 ما يجب تذكره من هذا الدرس

- البرنامج بلا **اختيارات** ينفذ دائمًا **سلسلة** العمليات نفسها في كل تنفيذ: وهذا ليس مفيدًا.

- يمثل **المخطط الانسيابي** الاختيارات **بمعيَّن** (سؤال صحيح/خطأ) وسهمين خارجين.
- `if (condizione) { … }` تنفذ الكتلة **فقط إذا** كان الشرط صحيحًا؛ و**الأقواس** إلزامية دائمًا.
- `if … else …` تضيف **الفرع** "وإلا": الكتلتان تستبعد إحداهما الأخرى دائمًا.
- انتبه إلى **الحدود**: `<` يستبعد القيمة الحدية، و`<=` يضمها. تطبق الآلة **الشرط** كما كُتب بالضبط، دون تفسير النيات.
- لأكثر من احتمالين نستخدم **التداخل** (`if` داخل `if`): فـ`if` متتاليتان غير **متداخلتين**، بشروط متراكبة، قد تنفذان عدة كتل معًا **بالخطأ**.
- يجب الحد من **التداخل** حتى لا تتضرر قابلية قراءة الكود.

مثال

للتحضير للدرس القادم: التدرب على **`if`** و`if-else` في القسم، ببناء **شروط** على أمثلة عددية (درجات، قياسات)، قبل تناول **التداخل** على عدة مستويات.

10 مسرد: كلمات إيطالية تُستخدم في القسم

- **voto**: درجة
- **condizione**: شرط
- **vero / falso**: صحيح / خطأ
- **sì / no**: نعم / لا
- **altrimenti**: وإلا (`else`)
- **ramo / blocco**: فرع / كتلة
- **annidamento**: تداخل
- **maggiore / minore**: أكبر / أصغر
- **Bene, Bravo, Ottimo**: جيد، أحسنت، ممتاز
- **Sufficiente**: مقبول
- **bocciato / promosso**: راسب / ناجح